Tarefa 16: Implementação de Produto Matriz-Vetor Usando MPI

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

A tarefa consiste em calcular $y = A\,x$, com A de tamanho $M \times N$ e x de tamanho N, usando MPI. A matriz é dividida por linhas com `MPI_Scatter`, o vetor x é enviado inteiro a todos com `MPI_Bcast`, e o resultado é reunido no processo 0 com `MPI_Gather`.

O objetivo é comparar os tempos para diferentes tamanhos de matriz e números de processos, e entender **quanto do tempo é gasto em comunicação**, principalmente no `Scatter` e no `Bcast`.

2 Metodologia

Ambiente. Código em C com MPI (Open MPI), executado no supercomputador NPAD, partição `amd-512`, em 2 nós alocados de forma exclusiva e submetido via SLURM.

Estratégia.

- O processo 0 cria A e x (valores determinísticos).
- `MPI_Scatter`: cada processo recebe M/P linhas de A.
- `MPI_Bcast`: todos recebem uma cópia completa de x.
- Cada processo calcula suas linhas de y; o `MPI_Gather` leva tudo ao processo 0.

Como as operações coletivas funcionam (visão geral).

- `MPI_Scatter`: um processo (o *root*) divide um buffer em P blocos iguais e entrega o bloco i ao processo i. Todos os dados de A *saem de um único processo*, então o volume que o root precisa mover é aproximadamente o tamanho de A inteiro, independente de P. O root também copia o seu próprio bloco para o buffer local.
- `MPI_Bcast`: o root envia o *mesmo* dado a todos. As implementações costumam usar uma árvore (por exemplo, binomial): quem já recebeu ajuda a repassar, de modo que são necessárias cerca de $\log_2 P$ etapas, e não $P - 1$ envios do root.
- `MPI_Gather`: operação inversa do `Scatter`; aqui só trafegam M valores de y.

$M = N$	A (`Scatter`)	x (`Bcast`)	y (`Gather`)
1024	8,4 MB	8,2 kB	8,2 kB
4096	134 MB	32,8 kB	32,8 kB
8192	537 MB	65,5 kB	65,5 kB
16384	2147 MB	131 kB	131 kB

Tabela 1: Volume de dados de cada operação coletiva (com $M = N$).

Volume de dados. O que muda entre as operações é o *volume*, não o número de chamadas (Tabela 1). A ocupa $8MN$ bytes, enquanto x e y ocupam apenas $8N$ e $8M$ bytes: A é N vezes maior que x.

Medição. Cada configuração foi repetida 5 vezes (média; em cada repetição vale o processo mais lento). Foram medidos:

- **tempo total**: `Scatter` + `Bcast` + cálculo + `Gather`;
- **tempo de cálculo**: apenas o laço de multiplicação;
- **comunicação** = total − cálculo. Inclui `Scatter`, `Bcast`, `Gather` e esperas entre processos.

Casos. Intra-nó: 1, 2, 4 e 8 processos em um único nó. Inter-nó: 2, 4, 8 e 16 processos distribuídos em 2 nós, metade em cada um. Tamanhos ($M = N$): 1024, 2048, 4096, 8192 e 16384.

3 Resultados

A Tabela 2 mostra o tempo total, e a Tabela 3 detalha o maior caso. A Figura 1 separa cálculo e comunicação para o menor e o maior tamanho.

	Tempo total (ms), $M = N$				
Caso	1024	2048	4096	8192	16384
intra, 1 proc	1,57	6,66	29,39	116,37	466,86
intra, 2 proc	4,50	8,25	28,17	103,68	351,47
intra, 4 proc	2,71	7,98	20,94	78,28	323,03
intra, 8 proc	2,38	8,09	19,42	71,40	302,23
inter, 2 proc	1,49	5,24	21,41	85,46	345,15
inter, 4 proc	4,32	11,26	23,41	84,71	312,44
inter, 8 proc	12,23	15,66	32,28	80,39	290,38
inter, 16 proc	5,74	8,10	15,14	42,53	130,99

Tabela 2: Tempo total em milissegundos.

Caso ($M = N = 16384$)	Total (ms)	Cálculo (ms)	Comun. (ms)	Comun. (%)	Speedup total	Speedup cálculo
intra, 1 proc	466,9	269,0	197,9	42%	1,00	1,00
intra, 2 proc	351,5	135,8	215,7	61%	1,33	1,98
intra, 4 proc	323,0	68,7	254,3	79%	1,45	3,91
intra, 8 proc	302,2	34,8	267,4	88%	1,54	7,73
inter, 2 proc	345,1	135,6	209,6	61%	1,35	1,98
inter, 4 proc	312,4	70,1	242,4	78%	1,49	3,84
inter, 8 proc	290,4	34,4	256,0	88%	1,61	7,82
inter, 16 proc	131,0	17,9	113,1	86%	3,56	15,00

Tabela 3: Detalhamento para $M = N = 16384$. Speedup em relação a 1 processo.

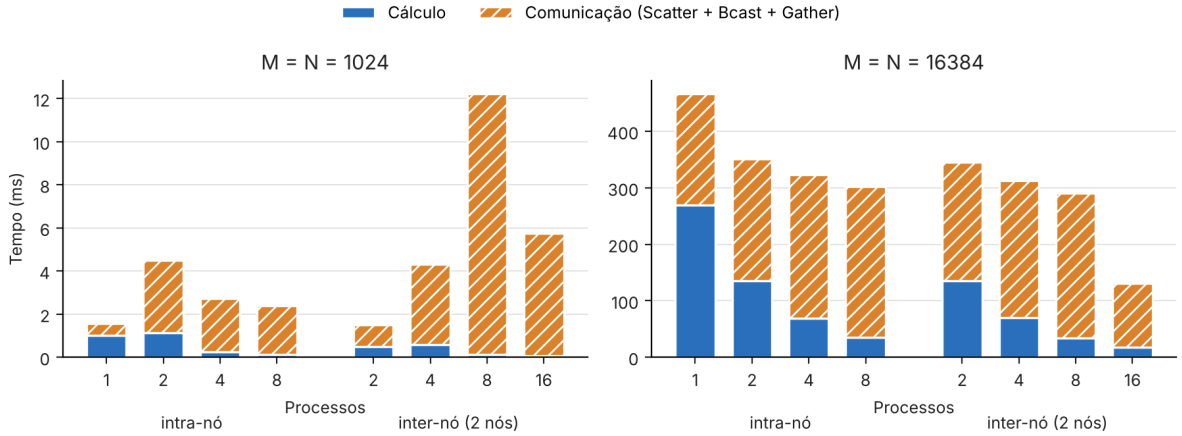


Figura 1: Tempo dividido entre cálculo e comunicação para $M = N = 1024$ e $M = N = 16384$.

4 Análise

O cálculo escala bem, o tempo total não. Para 16384, o speedup do cálculo é quase ideal (7,7 com 8 processos e 15,0 com 16), mas o do tempo total fica em apenas 1,5 com 8 processos no mesmo nó. A causa é a comunicação, que passa de 42% do tempo com 1 processo para 88% com 8.

Impacto do `MPI_Scatter`. É a operação dominante. Ela move A inteira (2,1 GB) a partir de um único processo, então o custo é limitado pela capacidade do root de copiar e enviar dados. Aumentar P divide o cálculo, mas *não reduz o volume que o root precisa distribuir*; por isso a comunicação ficou na faixa de 200 a 270 ms para 1 a 8 processos, e até cresceu levemente. Com 1 processo ela ainda custa 198 ms, o que é coerente com o root copiar A para o buffer local, mesmo sem outro processo para receber.

Impacto do `MPI_Bcast`. Em tamanhos grandes é desprezível: x tem 131 kB, cerca de 16 000 vezes menos dado que A. Como o `Bcast` usa uma árvore com $\log_2 P$ etapas (1, 2, 3 e 4 etapas para $P = 2, 4, 8, 16$), seu custo cresce devagar com P. Em tamanhos pequenos o quadro muda: para 1024 todos os volumes são pequenos, e o tempo passa a ser dominado pela *latência* e pela sincronização das coletivas (comunicação de 87% a 99% do tempo nos

casos com 4 ou mais processos). Nesse regime, adicionar processos *piora* o tempo total (por exemplo, 12,2 ms com 8 processos em 2 nós, contra 1,6 ms com 1 processo). O valor de 12,2 ms foge do padrão (16 processos deu 5,7 ms) e pode envolver ruído ou custos de inicialização da comunicação entre nós, já que a média tem só 5 repetições.

Intra-nó contra inter-nó. Com a mesma quantidade de processos, os tempos foram parecidos (por exemplo, 302 ms contra 290 ms com 8 processos em 16384), então a rede não foi um gargalo adicional visível para esses volumes. O destaque é `inter-nó` com 16 processos: o total caiu para 131 ms (speedup 3,56) e a comunicação para 113 ms, contra cerca de 267 ms com 8 processos em um nó.

5 Conclusão

- O ganho em tempo total é limitado pelo `MPI_Scatter` de A: o volume que um único processo distribui não diminui com mais processos, enquanto o cálculo diminui quase proporcionalmente.
- O `MPI_Bcast` de x é barato para matrizes grandes (pouco dado, custo em árvore $\sim \log_2 P$), mas a latência das coletivas domina para matrizes pequenas, onde usar mais processos pode piorar o tempo.
- A paralelização compensa para matrizes grandes e, mesmo assim, de forma parcial. O melhor resultado foi 3,56 vezes mais rápido com 16 processos em 2 nós ($M = N = 16384$).
- Como melhoria, evitar o `Scatter` gerando cada bloco de A diretamente em cada processo (cada um inicializa suas linhas) eliminaria a maior parte da comunicação.

A Código-Fonte

```c
/**TAREFA 16
 *
 * compilacao: mpicc -O3 -o tarefa16 tarefa16.c
 */

#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <mpi.h>

/**
 * INICIALIZACAO
 */
static void inicializa_matriz(double *A, int M, int N)
{
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            A[(size_t)i * N + j] = (double)((i + j) % 10 + 1);
}

static void inicializa_vetor(double *x, int N)
{
    for (int i = 0; i < N; i++)
        x[i] = (double)(i % 5 + 1);
}

/**
 * MAIN
 */
int main(int argc, char **argv)
{
    int rank, P;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &P);

    if (argc < 3) {
        if (rank == 0)
            fprintf(stderr, "Uso: %s <M> <N> [REPS]\n", argv[0]);
        MPI_Finalize();
        return 1;
    }

    int M = atoi(argv[1]);
    int N = atoi(argv[2]);
    int reps = (argc > 3) ? atoi(argv[3]) : 5;

    if (M <= 0 || N <= 0 || reps <= 0) {
        if (rank == 0)
            fprintf(stderr, "Parametros invalidos.\n");
        MPI_Finalize();
        return 1;
    }

    /* MPI_Scatter exige blocos iguais -- Padding se necessario */
```

```c
    int linhas_local = (M + P - 1) / P;
    int M_pad = linhas_local * P;

    double *A = NULL, *y = NULL;
    double *x = malloc((size_t)N * sizeof(double));
    double *A_local = malloc((size_t)linhas_local * N * sizeof(double));
    double *y_local = malloc((size_t)linhas_local * sizeof(double));

    if (!x || !A_local || !y_local) {
        fprintf(stderr,
                "Processo %d: falha de alocacao (x=%p A_local=%p y_local=%p | "
                "M=%d N=%d P=%d linhas_local=%d, A_local=%.1f MB)\n",
                rank, (void *)x, (void *)A_local, (void *)y_local,
                M, N, P, linhas_local,
                (double)linhas_local * N * sizeof(double) / 1e6);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    if (rank == 0) {
        A = calloc((size_t)M_pad * N, sizeof(double));
        y = malloc((size_t)M_pad * sizeof(double));
        if (!A || !y) {
            fprintf(stderr, "Processo 0: falha de alocacao de A/y.\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }

        inicializa_matriz(A, M, N);
        inicializa_vetor(x, N);
    }

    double soma_total = 0.0, soma_calc = 0.0;

    for (int r = 0; r < reps; r++) {
        MPI_Barrier(MPI_COMM_WORLD);
        double t0 = MPI_Wtime();

        /* Distribuiu linhas */
        MPI_Scatter(A, linhas_local * N, MPI_DOUBLE,
                    A_local, linhas_local * N, MPI_DOUBLE,
                    0, MPI_COMM_WORLD);

        /* Distribui x inteiro */
        MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

        /* Calculo */
        double tc0 = MPI_Wtime();
        for (int i = 0; i < linhas_local; i++) {
            double soma = 0.0;
            for (int j = 0; j < N; j++)
                soma += A_local[(size_t)i * N + j] * x[j];
            y_local[i] = soma;
        }
        double tc1 = MPI_Wtime();

        /* Coleta no processo 0 */
        MPI_Gather(y_local, linhas_local, MPI_DOUBLE,
                   y, linhas_local, MPI_DOUBLE,
                   0, MPI_COMM_WORLD);
```

```
115
116         double t1 = MPI_Wtime();
117
118         /* tempo do experimento */
119
120         double t_total_loc = t1 - t0, t_calc_loc = tc1 - tc0;
121         double t_total, t_calc;
122
123         MPI_Reduce(&t_total_loc, &t_total, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
124         MPI_Reduce(&t_calc_loc, &t_calc, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
125
126         if (rank == 0) {
127             soma_total += t_total;
128             soma_calc += t_calc;
129         }
130     }
131
132     if (rank == 0) {
133         printf("%d,%d,%d,%.6f,%.6f\n", P, M, N, soma_total / reps, soma_calc / reps);
134
135         free(A);
136         free(y);
137     }
138
139     free(x);
140     free(A_local);
141     free(y_local);
142
143     MPI_Finalize();
144     return 0;
145 }
```

Código 1: Produto matriz-vetor com MPI (`tarefa16.c`).